

UNIT-5-Cursors and Exception Handling

5.1 Concepts of Cursors

- Oracle creates memory area where all internal processing related data are stored for execution of SQL statements. It is known as "Context Area or Work Area". Oracle Engine fetches the data from this area.
- This work area is private to SQL's operations and it is also called CURSOR.
- The data which is stored in Cursor is called Active Data Set.
- The size of cursor is depended on the size of the Active Data Set.
- The value which is retrieved from Tables is held in Cursor which is processed by Oracle Engine for further processing.
- Multiple cursors can also be hold by Oracle at a time

Advantages of using Cursor:

- Cursor allows to access data row-wise, so validation or any kind of operation can easily be implemented on it.
- Query response time is increasing because respective data is available in Active Data Set.
- Concurrency control (Multiple user using same data at a same time) can be achieved properly using Cursor.
- Cursor Query processing is fast as compare to Loop.

Disadvantages of using Cursor:

- Cursor occupies space in system memory which can be used for other purpose.
- It decreases the performance of Insert, Update and Delete statement.

5.1.1 Types of cursors

- There are two types of cursors: Implicit & Explicit

Implicit Cursor:

- If the cursor is created automatically by Oracle for internal processing, then it is known as Implicit Cursor. It is created when the query is executed.
- Implicit cursor holds rows about last executed DML statement (insert, update, delete).
- Implicit Cursor attributes (%FOUND,%NOTFOUND,%ISOPEN,%ROWCOUNT) can be used to check the status of applied DML statement. These attributes can be used using implicit cursor name (SQL). Ex: SQL%found
- These attributes help in checking whether any rows are affected by implementing above mentioned statement or not.

- If single row is not affected then in place of generating error it will set TRUE or FALSE (Boolean value) in specific attribute which helps for generating user defined error message.
- Programmer cannot control the implicit cursor.

Example:

```
begin
    update stud_info set marks=40 where rno=&rno;
    if SQL%found then
        dbms_output.put_line(SQL%rowcount || ' rows modified');
    else
        dbms_output.put_line('Data not found');
    end if;
end;
/
```

```
Enter value for rno: 2
old 2: update stud_info set marks=40 where rno=&rno;
new 2: update stud_info set marks=40 where rno=2;
1 rows modified
```

```
Enter value for rno: 7
old 2: update stud_info set marks=40 where rno=&rno;
new 2: update stud_info set marks=40 where rno=7;
Data not found
```

Explicit Cursor:

- When the cursor is created through PL/SQL block by user for external use then it is known as Explicit Cursor. It is also known as User Defined Cursor.
- Steps for explicit cursor:
 1. Declare cursor in DECLARE section and map it to SQL SELECT statement.
 2. OPEN the cursor.
 3. FETCH the data from cursor (one row at a time). Process data using loop.
 4. CLOSE the cursor

5.1.2 Declare, open, fetch and close cursors.

Declaration of Cursor:

- Cursor is declared in DECLARE section of PL/SQL.
- The name is given to the cursor and the cursor mapping is done with Specified SELECT query.
- The memory allocation is not done while declaring cursor. For that, there is a need to open cursor.
- Syntax:


```
CURSOR cur_name IS SELECT statement;
```
- Example:


```
CURSOR cstud IS select rno, name, marks from stud_info;
```

Open Cursor:

- Cursor OPEN statement is written in BEGIN section of PL/SQL.
- Initialization of Cursor is done via OPEN statement. It performs following tasks:
 - o Allocate SQL work area to this cursor name.
 - o Execute query associated with cursor name.
 - o Retrieve data and keep it in assigned work area (Active data set).
 - o Set the pointer to the first record of the Active data set.
- OPEN statement makes cursor ready to fetch data.
- Syntax:


```
OPEN cur_name;
```
- Example:


```
OPEN cstud;
```

Fetch Data using Cursor:

- FETCH statement is written in BEGIN section with LOOP statement of PL/SQL.
- FETCH statement is used to retrieve the record from the active data set using memory variables written in PL/SQL block.
- It processes one row at a time.
- When one FETCH statement is executed, the CURSOR pointer is moved to the next record in Active Data Set.
- LOOP-END LOOP is used to fetch the data row-wise (One row at a time) from Active Data Set.
- Syntax:


```
FETCH cur_name INTO var1, var2, ....;
```
- Ex:


```
FETCH cstud INTO rno, name, marks;
```

Close Cursor:

- CLOSE statement is written in BEGIN section.
- CLOSE statement closes the current cursor.
- It removes the active data set and releases the work area assigned to Cursor.
- Syntax:


```
CLOSE cur_name;
```
- Ex:


```
CLOSE cstud;
```

Example: Display students' detail using Cursor.

```
declare
  rno stud_info.rno %type;
  rname stud_info.name %type;
```

```

rmarks stud_info.marks %type;
cursor cstud is select rno,name,marks from stud_info;
begin
    open cstud;
    loop
        fetch cstud into rrno, rname, rmarks;
        exit when cstud%notfound;
        dbms_output.put_line('Rollno: ' || rrno || ' Name: ' || rname || ' Marks: ' || rmarks);
    end loop;
    close cstud;
end;
/

```

```

Rollno: 1 Name: Mayank Marks: 35
Rollno: 2 Name: Maya Marks: 25
Rollno: 3 Name: Mohit Marks: 45
Rollno: 4 Name: Raghav Marks: 10
Rollno: 5 Name: Ankit Marks: 49

```

Example: Insert the records of department1 into another table: tblstud_d1(rno, name, marks)

```

create table tblstud_d1(rno int, name varchar2(10), marks int);

declare
    r stud_info %rowtype;
    cursor c is select rno,name,marks from stud_info where did=1;
begin
    open c;
    loop
        fetch c into r.rno,r.name,r.marks;
        exit when c%notfound;
        insert into tblstud_d1 values(r.rno,r.name,r.marks);
    end loop;
    close c;
end;
/

Select * from tblstud_d1;

```

RNO	NAME	MARKS
1	Mayank	35
3	Mohit	45

Using FOR loop: Example: Display students' detail whose name start with letter 'M'.

```
declare
    cursor cs is select * from stud_info where name like 'M%';
begin
    for r in cs loop
        dbms_output.put_line('Rollno: ' || r.rno || ' Name: ' || r.name || ' Marks: ' || r.marks);
    end loop;
end;
/
```

```
Rollno: 1 Name: Mayank Marks: 35
Rollno: 2 Name: Maya Marks: 25
Rollno: 3 Name: Mohit Marks: 45
```

5.1.3 Cursor Attributes:

- When cursor is created then automatically cursor attributes are created by Oracle.
- These are used to keep track of Current status of cursor.
- These variables are used through PL/SQL block.
- Following are the attributes:

Implicit cursor attributes:

Attributes	Description
%ISOPEN	SQL%ISOPEN: Oracle engine automatically open and close SQL cursor after executing DML Statement. So SQL%isopen cannot be referenced outside SQL statement. So it always returns FALSE.
%FOUND	SQL%FOUND It returns TRUE if atleast one row is available in cursor. It returns FALSE if not a single row is available in cursor.
%NOTFOUND	SQL%NOTFOUND It returns TRUE if not a single row is available in cursor. It returns FALSE if atleast one row is available in cursor.
%ROWCOUNT	SQL%ROWCOUNT It returns the total numbers of records affected after executing statement.

Explicit cursor attributes:

Attributes	Description
%ISOPEN	CUR_NAME%ISOPEN: It returns TRUE if explicit cursor is open. It returns FALSE if explicit cursor is not open.
%FOUND	CUR_NAME %FOUND

	It returns TRUE if atleast one row is available in cursor. It returns FALSE if not a single row is available in cursor.
%NOTFOUND	CUR_NAME %NOTFOUND It returns TRUE if not a single row is available in cursor. It returns FALSE if atleast one row is available in cursor.
%ROWCOUNT	CUR_NAME %ROWCOUNT It returns the total numbers of records affected after executing statement.

5.2 Exception Handling in PL/SQL

- Exception is an error condition during a program execution means during run time.
- PL/SQL allows programmer to solve the run-time error through program block.
- It allows catching the error and solving this error using EXCEPTION block of the program during program execution.
- Syntax:

```

DECLARE
    <declaration section>
BEGIN
    <executable statements>
EXCEPTION
    <exception handling block>
    WHEN exception1 THEN
        <exception handling statements>
    WHEN exception1 THEN
        <exception handling statements>
    ....
    WHEN others THEN
        <exception handling statements>
END;
```

- PL/SQL executes <executable statements> and when any error occurs, it searches for respective exception handler from EXCEPTION section and performs the respective <exception handling statements>.
- OTHERS' exception handling statements are executed when the error occurs which is not externally solved by mentioned exception handlers.
- There are two types of exceptions available:
 - System Exceptions:
 - User-defined Exceptions

5.2.1 Types of Exceptions:

5.2.1.1 Named System Exceptions

- It is a part of System Exception.
- System exceptions are automatically raised by Oracle when any program violates RDBMS rules. These system exceptions are known as Named System Exceptions where specific pre-defined names are given to the errors.

- These types of errors occur frequently.
- Ex: NO_DATA_FOUND, ZERO_DIVIDE
- There is no need to declare these named system exceptions explicitly.
- These exceptions are raised implicitly when any pre-defined error occurs.
- The respective error handling code can be written within program EXCEPTION block.
- List of Named System Exceptions are:

Exception	Oracle Error	SQL Code	Description
ACCESS_INTO_NULL	06530	-6530	It is raised when a NULL object is automatically assigned a value.
CASE_NOT_FOUND	06592	-6592	It is raised when none of the choices in the "WHEN" clauses of a CASE statement is selected, and there is no else clause.
COLLECTION_IS_NULL	06531	-6531	It is raised when a program attempts to apply collection methods other than exists to an uninitialized nested table or varray, or the program attempts to assign values to the elements of an uninitialized nested table or varray.
DUP_VAL_ON_INDEX	00001	-1	It is raised when duplicate values are attempted to be stored in a column with unique index.
INVALID_CURSOR	01001	-1001	It is raised when attempts are made to make a cursor operation that is not allowed, such as closing an unopened cursor.
INVALID_NUMBER	01722	-1722	It is raised when the conversion of a character string into a number fails because the string does not represent a valid number.
LOGIN_DENIED	01017	-1017	It is raised when a program attempts to log on to the database with an invalid username or password.
NO_DATA_FOUND	01403	+100	It is raised when a select into statement returns no rows.
NOT_LOGGED_ON	01012	-1012	It is raised when a database call is issued without being connected to the database.
PROGRAM_ERROR	06501	-6501	It is raised when PL/SQL has an internal problem.
ROWTYPE_MISMATCH	06504	-6504	It is raised when a cursor fetches value in a variable having incompatible data type.
SELF_IS_NULL	30625	-30625	It is raised when a member method is invoked, but the instance of the object type was not initialized.
STORAGE_ERROR	06500	-6500	It is raised when PL/SQL ran out of memory or memory was corrupted.

TOO_MANY_ROWS	01422	-1422	It is raised when a SELECT INTO statement returns more than one row.
VALUE_ERROR	06502	-6502	It is raised when an arithmetic, conversion, truncation, or size-constraint error occurs.
ZERO_DIVIDE	01476	1476	It is raised when an attempt is made to divide a number by zero.

- Example: (Display student's detail whose marks are more than user' specified marks)

```

declare
    r stud_info %rowtype;
begin
    select * into r from stud_info where marks >&marks;
    dbms_output.put_line('Rollno: ' || r.rno);
    dbms_output.put_line('Name: ' || r.name);
exception
    when no_data_found then
        dbms_output.put_line('Rollno does not exists');
    when too_many_rows then
        dbms_output.put_line('Multiple rows as output');
    when others then
        dbms_output.put_line('Found error.....');
end;
/

```

```

Enter value for marks: 48
old  4: select * into r from stud_info where marks >&marks;
new  4: select * into r from stud_info where marks >48;
Rollno: 5
Name: Ankit

```

```

Enter value for marks: 60
old  4: select * into r from stud_info where marks >&marks;
new  4: select * into r from stud_info where marks >60;
Rollno does not exists

```

```

Enter value for marks: 10
old  4: select * into r from stud_info where marks >&marks;
new  4: select * into r from stud_info where marks >10;
Multiple rows as output

```

5.2.1.2 Unnamed System Exceptions

- It is a part of System Exception.

- System exceptions for which the Oracle doesn't specify any pre-defined name are known as Unnamed System Exception.
- These types of errors are not occur frequently.
- These exceptions don't have any error_name but they have error_number with associated message.
- Error numbers are the negative numbers under the range -20000 to -20999 which are assigned by Exception Handler.
- PRAGMA EXCEPTION INIT() is used to bind a numbered exception handler to name. This process is handled in DECLARE section.

Syntax: PRAGMA EXCEPTION INIT(exception_name, error_number)

When any error occurs then the respective error_number is bind with mentioned exception_name which is used in EXCEPTION section for handling this error externally.

When it is done, the default exception handling code of exception handler is overridden by user defined exception handling code.

Example: PRAGMA EXCEPTION INIT(exp1,-20111);

- Syntax:

```
DECLARE
    exception_name EXCEPTION;
    PRAGMA EXCEPTION INIT(exception_name, Error_code);
BEGIN
    ...
EXCEPTION
    WHEN exception_name THEN
        Action_code
END;
```

- Example: Delete record of Department 1 from tbldept table. (Parent Table)

```
declare
    parent_delete exception;
    pragma exception_init(parent_delete,-02292);
begin
    delete from tbldept where dno=1;
    dbms_output.put_line('Deleted');
exception
    when parent_delete then
        dbms_output.put_line('Not Possible to delete');
end;
/
```

5.2.1.3 User-defined Exceptions

- During system creation, there are some of the Business Rules which need to follow by system. In this type of rules, system doesn't generate any error. So for solving this type of situation, user can create user defined exceptions.
- User defined exceptions are declared in DECLARE section.
- The error should be raised used RAISE keyword within BEGIN section.
- RAISED exception can be handled within EXCEPTION section.
- Syntax:

```
DECLARE
    exception_name EXCEPTION;
BEGIN
    ...
    IF condition THEN
        RAISE exception_name;
    END IF;
EXCEPTION
    WHEN exception_name THEN
        Action_code;
END;
```

- Example: If mentioned rollno is negative display error message and display the detail of students if rollno is available.

```
declare
    invalid_input exception;
    r int :=&r;
    s stud_info %rowtype;
begin
    if r<0 then
        raise invalid_input;
    end if;
    select * into s from stud_info where rno=r;
    dbms_output.put_line('Rollno: ' || s.rno);
    dbms_output.put_line('Name: ' || s.name);
exception
    when invalid_input then
        dbms_output.put_line('Rollno should be positive integer number.');
```

end;
/